

PATTERNBEE: ENABLING ZIGBEE-TO-BLE DIRECT COMMUNICATION BY OFFSET RESISTANT PATTERNS

Shumin Yao, Li Feng, Jing Zhao, Qinglin Zhao, Qiyu Yang, and Wenchao Jiang

ABSTRACT

In Internet of Things (IoT), ZigBee and Bluetooth Low Energy (BLE) are two popular technologies. Their devices often coexist and enabling them to communicate with each other will facilitate our daily lives greatly. In this article, we present PatternBee, a lightweight physical-level cross-technology communication (PHY-level CTC) scheme which enables ZigBee-to-BLE direct communications. In PatternBee, the BLE receiver divides received ZigBee symbols into pieces. We find that some pieces always have a phase shift of 0 no matter how large the sampling offset is. We call these pieces offset resist pieces (ORPs). By identifying the occurrence pattern of these ORPs, we propose a Pattern-based decoding algorithm. With this algorithm, the BLE receiver can decode ZigBee symbols at very low processing complexity, while achieving very high decoding accuracy. Real-world experiments verify the effectiveness and accuracy of PatternBee.

INTRODUCTION

With the increasing popularity of Internet of Things (IoT), a huge set of diversified wireless technologies come into being [1]. These heterogeneous technologies coexist on unlicensed industrial scientific medical (ISM) bands and therefore may lead to cross-technology interference and network performance degradation. Nevertheless, each of these technologies is specialized and hence may possess strengths in some areas, but weaknesses in the others. Therefore, interconnecting heterogeneous IoT devices (i.e., the devices adopting heterogeneous technologies), while fully utilizing their respective advantages to facilitate our daily lives, is a crucial but challenging issue.

The traditional approach of achieving interconnection between heterogeneous devices is to deploy an IoT gateway that is equipped with multiple wireless network interface cards (NICs) [2]. In this approach, the gateway needs to perform protocol conversions. That is, it first receives a data packet from the sender via one NIC and then passes the packet to the upper layer, where it converts the received packet to a new receiver compliant packet. Next, it passes the new packet to the lower layer and transmits it via another NIC. However, this approach introduces extra hardware cost, protocol conversion complexity, traffic overhead, and end-to-end relay delay [3].

Recently, researchers have proposed a novel cross-technology communication (CTC) approach to overcome the above drawbacks. This approach enables direct communications between heterogeneous devices. There are two types of CTCs.

One type of CTC is the packet-level CTC. It uses packet level information to convey messages across technologies. As shown in Fig. 1a, it may adopt an energy-level-based scheme [4] (i.e., bits are conveyed by changing packet energy levels), a packet-interval-based scheme [5] (i.e., bits are conveyed by changing packet transmission intervals), and a packet-reorder-based scheme [6] (i.e., bits are conveyed by shifting packet transmission orders). Unfortunately, this technology requires both the sender and receiver to modify their respective modulation and demodulation methods, which is infeasible for low-end (i.e., resource-limited) IoT devices. It also has intrinsic limitations in the data rate, because it only conveys a few bits per packet.

Another type of CTC is the physical-level (PHY-level) CTC. It uses PHY-layer information to convey messages across technologies. In particular, it carries bits per PHY-layer symbol and therefore can achieve far higher throughput than the packet-level CTC. As shown in Fig. 1b, existing PHY-level CTC designs can be categorized into three types of schemes, where parameters (e.g., M , T_1 and R_1) will be detailed later. The first is the transmitter-side (TX-side) scheme [7], which enables a transmitter to emulate the receiver-compliant waveform so that the receiver can receive the signal as usual. The second is the receiver-side (RX-side) scheme [8], in which a transmitter operates as usual, but the receiver adds some additional software modules to decode the transmitter's signal directly. The third is the both-side scheme [9, 10], in which both the transmitter and the receiver add some additional modules so as to convey information via mutually agreed waveforms.

The RX-side PHY-level CTC transfers the processing complexity to the receiver. XBee [8] is a typical representative of such a CTC. It achieves ZigBee-to-BLE direct communications. ZigBee and BLE are two popular technologies for IoT, and their direct communications will trigger many practical applications in smart home, smart factory, indoor navigation, and so on [8]. For example, a smart BLE speaker may play temperature information from smart devices with ZigBee radio. However, this

Shumin Yao, Li Feng (corresponding author), Jing Zhao, and Qinglin Zhao are with Macau University of Science and Technology; Qiyu Yang is with Guangdong University of Technology, also with the Joint International Research Laboratory of Intelligent Information Processing and System Integration of IoT, Ministry of Education of the P.R.C., and also with the Guangdong-HongKong-Macao Joint Laboratory for Smart Discrete Manufacturing; Wenchao Jiang is with Singapore University of Technology and Design.

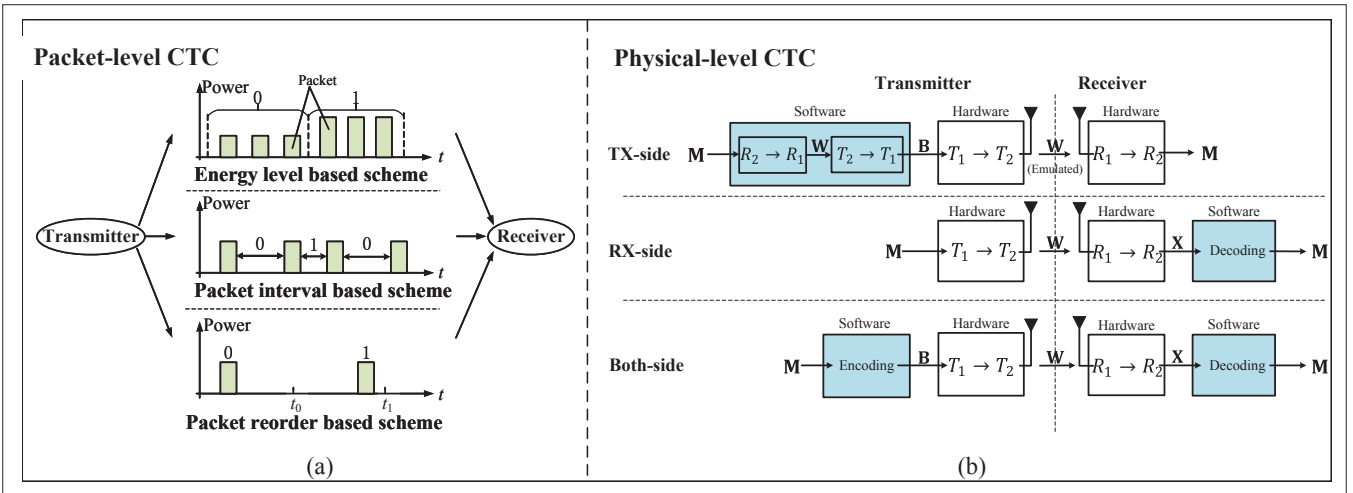


FIGURE 1. a) Packet-level CTC; b) physical-level CTC.

scheme adopts an exhaustive correlation method for decoding and therefore leads to high processing complexity. In this scheme, whenever receiving a ZigBee symbol, the BLE receiver correlates the received symbol with each of all defined ZigBee symbols, and regards the symbol (corresponding to the maximum correlation value) as the received symbol.

For ZigBee-to-BLE direct communications, XBee requires the BLE receiver to be capable of executing high-intensity computational tasks such as exhaustive correlation. This will run out of the receiver's energy quickly and therefore limit its applications greatly. This motivates us to propose a lightweight RX-side CTC without correlation. However, enabling a BLE receiver to identify ZigBee symbols without correlation is a great challenge, because of the non-orthogonality of ZigBee symbols and the heterogeneity between BLE and ZigBee.

In this article, we propose PatternBee to meet this challenge. In PatternBee, the BLE receiver divides each received ZigBee symbol into a series of pieces. In a practical system, the sampling offset is inevitable. However, we find that some pieces always have a phase shift of 0 no matter how large the offset is and therefore define such pieces as offset resist pieces (ORPs). By identifying the occurrence pattern of these ORPs (instead of performing correlation), we can decode the received ZigBee symbol quickly by checking its pieces only, instead of correlating it and each of all 16 ZigBee symbols. In this way, our scheme achieves a very low processing complexity and at the same time achieves a very high decoding accuracy.

The rest of the article is organized as follows. We present the PHY-level CTC in the next section, and then specifically outline existing RX-side PHY-level CTC designs. We then propose the PatternBee, after which we detail our pattern-based decoding algorithm. We evaluate the performance of PatternBee, and finally conclude this article in the final section.

PHY-LEVEL CTC

PHY-level CTC designs can be divided into three types: transmitter-side (TX-side), receiver-side (RX-side), and both-side schemes. Without loss of generality, we make the following assumptions.

As shown in Fig. 1b, for conventional transmission, the transmitter feeds bits B into its TX hardware, where B is transformed into a waveform W after being processed sequentially by two operations: T_1 and T_2 (e.g., whitening and modulation). For conventional reception, the receiver feeds the received W into its RX hardware, where W is decoded into bits after being processed sequentially by two operations, R_1 and R_2 (e.g., demodulation and dewhitening).

Now, to enable direct communications between two heterogeneous devices, PHY-level CTC technology adds software-defined PHY modules that utilize the characteristics of conventional hardware transmission and reception operations. Below, assuming that a transmitter desires to send a message M to a heterogeneous receiver, we explain the TX-, RX-, and both-side schemes.

TX-SIDE SCHEME

In the TX-side scheme, the basic idea is to enable the transmitter to emulate the receiver-compliant waveform so that the receiver can receive the signal as usual.

With the help of Fig. 1b, we outline the implementation process of this idea. The transmitter adds an additional software module, which first implements the reverse RX operations (i.e., $R_2 \rightarrow R_1$) of the heterogeneous receiver hardware and then the reverse TX operations (i.e., $T_2 \rightarrow T_1$) of the transmitter hardware. As a result, to transmit a message M , the software module first transforms M to the receiver-compliant waveform W and then transforms W to a bit sequence B . Next, the transmitter's native hardware follows the conventional transmission process to restore the receiver-compliant waveform W according to B . Finally, the receiver's native hardware follows the conventional reception process to restore the original message M from W .

RX-SIDE SCHEME

The RX-side scheme transfers the processing complexity to the receiver. Therefore, the transmitter sends messages as usual, but the heterogeneous receiver adds an additional software module to decode the transmitter's signal directly.

With the help of Fig. 1b, we outline the implementation process of this idea. The transmitter

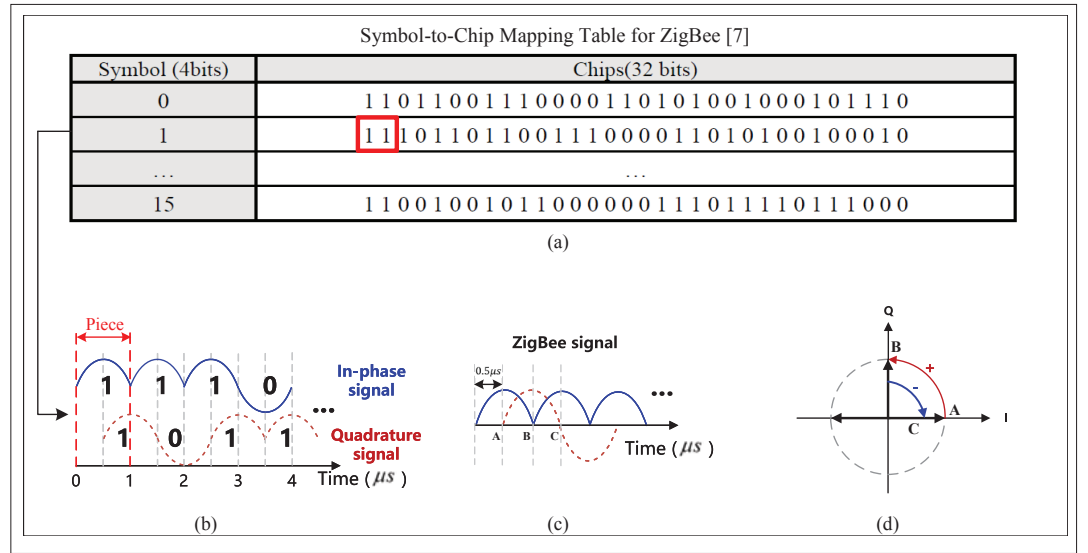


FIGURE 2. a) Symbol-to-Chip mapping table; b) in-phase signal and quadrature signal of ZigBee symbol "1;" c) the merged signal of ZigBee symbol "1;" d) the merged signal yields a $\pm\pi/2$ phase shift every 0.5 μ s

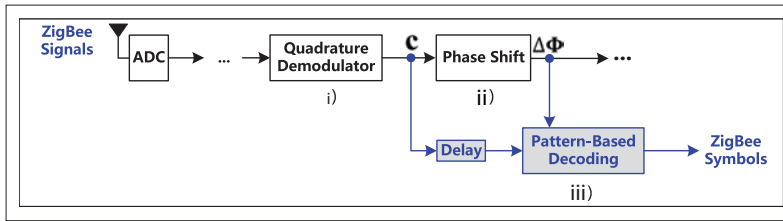


FIGURE 3. The workflow of the PatternBee receiver.

transforms a message $\mathbf{M}$ to its compliant waveform $\mathbf{W}$ via its native hardware, while the receiver transforms $\mathbf{W}$ to a bit sequence $\mathbf{X}$. The $\mathbf{X}$ is then fed into the introduced software module. In this module, the message $\mathbf{M}$ is decoded by emulating or utilizing the similarity between the transmitter's (de) modulation and the receiver's (de)modulation for decoding (which is explained below).

BOTH-SIDE SCHEME

In the both-side scheme, the transmitter generates mutually agreed waveforms with some features, while the receiver decodes messages based on these features.

With the help of Fig. 1b, we outline the implementation process of this idea. The transmitter uses an additional software module to transform a message $\mathbf{M}$ to a bit sequence $\mathbf{B}$ and then passes $\mathbf{B}$ to its hardware, which generates a mutually agreed waveform $\mathbf{W}$ with feature $\mathbf{X}$ (e.g., phase shifts). The receiver detects $\mathbf{W}$ using its hardware and obtains $\mathbf{X}$, and then it adds an additional software module to transform $\mathbf{X}$ to $\mathbf{M}$.

EXISTING RX-SIDE CTC DESIGNS

The RX-side CTC aims to enable low-end (i.e., resource-limited) to high-end communications. Therefore, it transfers the processing complexity to the receiver. LEGO-Fi [11] and XBee [8] are two of the latest representatives of RX-side CTC schemes.

LEGO-Fi enables ZigBee-to-WiFi direct communications. In LEGO-Fi, in order to decode ZigBee's signal, the WiFi receiver adds software modules

to emulate ZigBee's demodulation process. Specifically speaking, the WiFi receiver first reduces its sampling rate (i.e., extracts one from every five samples that are obtained from its native RX hardware) to be consistent with that of ZigBee. Then, it calculates the phase shift between two consecutive extracted samples by reusing its native quadrature demodulator. Next, it maps the obtained phase shifts to ZigBee symbols and decodes ZigBee signals. Different from LEGO-Fi, this article focuses on ZigBee-to-BLE communications.

XBee enables ZigBee-to-BLE direct communications, which is also a focus in this article. It utilizes the similarity between ZigBee and BLE modulations for decoding. ZigBee defines n symbols, each consisting of $2n$ -bit, where $n = 16$. Upon receiving a $2n$ -bit ZigBee symbol with noise (say, from a wireless channel), the BLE receiver first samples the symbol via its native RX hardware (using a half sampling rate of ZigBee) and then calculates the phase shift of two consecutive samples and finally transforms these phase shifts to an n -bit sequence. Adopting this approach, for each of n ideal $2n$ -bit ZigBee symbols (i.e., those without noise), XBee can obtain one n -bit ideal transformed sequence. We note ZigBee adopts offset quadrature phase shift keying (PSK) modulation and BLE adopts Gaussian frequency shift keying (FSK) modulation. FSK is intrinsically tied with PSK since the frequency is the time derivative of phase [12]. Therefore, each n -bit ideal transformed sequence can well be mapped to its corresponding ZigBee symbol. Benefiting this similarity of the two modulations, XBee adds a software module to make a correlation between the outputted n -bit sequence of the BLE receiver and each of n n -bit ideal transformed sequences and then de-map the ideal transformed sequence (corresponding to the maximum correlation value) to its corresponding ZigBee symbol.

PROPOSED ZIGBEE-TO-BLE CTC DESIGN

We propose a novel ZigBee-to-BLE CTC design called PatternBee, which decodes CTC messages based on offset resistant patterns at the receiver

side. Below, we introduce PatternBee's transceiving process.

PATTERNBEE TRANSMITTER

PatternBee adopts the off-the-shelf ZigBee transmitter to transmit ZigBee symbols. The transmitter first modulates symbols into ZigBee signals. Then, it amplifies and finally sends out these signals. Here, we only focus on the modulation part.

In modulation, the transmitter first adopts the direct sequence spread spectrum (DSSS) technique to transform a ZigBee symbol into a 32-chip sequence where one chip is one binary bit, according to the mapping table in Fig. 2a. Then, it adopts the offset quadrature phase-shift keying (OQPSK) technique to transform the 32-chip sequence to a ZigBee signal. With the help of Fig. 2b, taking ZigBee symbol "0" as an example, we explain the transformation processes in the following steps:

- The transmitter selects odd and even chips of the 32-chip sequence to construct an in-phase sequence and a quadrature sequence, respectively.
- The transmitter transforms the in-phase sequence and the quadrature sequence to an in-phase signal (shown in the blue curve) and a quadrature signal (shown in the red dash curve), respectively. Specifically speaking, it transforms each chip of sequences to a half-sine wave with length 1 μ s; the wave is positive if the chip is "1" and is negative otherwise.
- The transmitter delays the quadrature signal by 0.5 μ s and then merges it with the in-phase signal to generate a ZigBee signal.

Phase-Shift Property of ZigBee Signals: The merged signal yields a $\pm\pi/2$ phase shift every 0.5 μ s [13]. As shown in Fig. 2c, The in-phase and quadrature signal value pair (I, Q) = (1, 0) at time A = 0.5s, and (0, 1) at time B = 1s, and (1, 0) at time C = 1.5 μ s. Mark these I/Q pairs in the constellation diagram (Fig. 2d). We can see that the ZigBee signal has a $\pi/2$ phase shift from time A to B and a $-\pi/2$ phase shift from time B to C. Then, the ZigBee transmitter can convey chip "1" by a phase shift of $\pi/2$ and chip "0" by $-\pi/2$.

PATTERNBEE RECEIVER

PatternBee leaves the complexity of implementing ZigBee-to-BLE direct communication to the receiver and reuses the standard BLE module for ZigBee reception. The heart of the PatternBee receiver is our newly introduced pattern-based decoding algorithm. This algorithm, which is detailed later, decodes ZigBee symbols utilizing the phase-shift property of ZigBee's transmitting signals. Here, with the help of Fig. 3, we present the workflow of decoding a ZigBee symbol by the PatternBee receiver through the following three steps:

- The receiver reuses the BLE quadrature demodulator (Fig. 3i) to sample the received ZigBee signals with interval 1 μ s, and obtains a sequence of complex samples $\mathbf{C} = \{(I_i, Q_i)\}_{i=1}^{17}$, where (I_i, Q_i) is the k -th in-phase and quadrature signal value pair.
- Then, it reuses the BLE phase shift module (Fig. 3ii) to obtain a set of phase shifts $\Delta\Phi = \{\Delta\Phi_i\}_{i=1}^{16}$, where $\Delta\Phi = \angle(I_{i+1}, Q_{i+1}) - \angle(I_i, Q_i)$ is the i -th phase shift, and $\angle(I_i, Q_i)$ is the phase value of (I_i, Q_i) .

two-chip piece ZigBee Symbol value	(index)															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	11	01	10	01	11	00	00	11	01	01	00	10	00	10	11	10
1	11	10	11	01	10	01	11	00	00	11	01	01	00	10	00	10
2	00	10	11	10	11	01	10	01	11	00	00	11	01	01	00	10
3	00	10	00	10	11	10	11	01	10	01	11	00	00	11	01	01
4	01	01	00	10	00	10	11	10	11	01	10	01	11	00	00	11
5	00	11	01	01	00	10	00	10	11	10	11	01	10	01	11	00
6	11	00	00	11	01	01	00	10	00	10	11	10	11	01	10	01
7	10	01	11	00	00	11	01	01	00	10	00	10	11	10	11	01
8	10	00	11	00	10	01	01	10	00	00	01	11	01	11	10	11
9	10	11	10	00	11	00	10	01	01	10	00	00	01	11	01	11
10	01	11	10	11	10	00	11	00	10	01	01	10	00	00	01	11
11	01	11	01	11	10	11	10	00	11	00	10	01	01	10	00	00
12	00	00	01	11	01	11	10	11	10	00	11	00	10	01	01	10
13	01	10	00	00	01	11	01	11	10	11	10	00	11	00	10	01
14	10	01	01	10	00	00	01	11	01	11	10	11	10	00	11	00
15	11	00	10	01	01	10	00	00	01	11	01	11	10	11	10	00

FIGURE 4. Pieces 1 to 16, all full-ORPs (marked by red solid boxes) for all ZigBee symbols, and some semi-ORPs (marked by red circles) of ZigBee symbols "3" and "11."

- Finally, it feeds $\mathbf{C}$ and $\Delta\Phi$ into the newly introduced Pattern-based decoding module (Fig. 3iii) to decode the received ZigBee symbols. Here, it synchronizes $\mathbf{C}$ and $\Delta\Phi$ by slightly delaying $\mathbf{C}$ according to the processing time of the BLE phase shift module.

Remarks: This article mainly presents how a PatternBee receiver decodes ZigBee signals by offset resistant patterns. Regarding other practical issues (such as how a PatternBee receiver differentiates ZigBee signals from BLE signals and how a PatternBee receiver manages BLE channel hopping), please refer to XBee [8].

PATTERN-BASED DECODING

In this section, we first define piece and offset resistant piece (ORP), and reveal the pattern of ORRs, and finally present the pattern-based decoding algorithm.

PIECE

A PatternBee receiver reuses the standard BLE receiver to sample ZigBee signals, as shown in Fig. 3. Because the bandwidth of BLE (1MHz) is half that of ZigBee (2MHz), the sampling interval of the PatternBee receiver (1 μ s) is twice that of the ZigBee receiver (0.5 μ s). We define a piece to be one two-chip ZigBee signal segment with 1 μ s. In Fig. 2b, we show a piece corresponding to the two chips "11" of the ZigBee symbol "1." Note that for each ZigBee symbol, we have 32 chips and hence there are 16 pieces (Fig. 4). Further, the phase shift of a piece with 1 μ s is the accumulation of two consecutive ZigBee phase shifts, because ZigBee signals have a phase shift of $\pm\pi/2$ every 0.5 μ s according to the phase-shift property.

OFFSET RESISTANT PIECE (ORP)

In a practical CTC system, it is unavoidable to introduce a sampling offset (denoted as Δt), because of the unsynchronized transmitter and receiver or the signal distortion in the air. We find

While PatternBee achieves almost the same throughput as XBee, it has much lower decoding time complexity and energy consumption than XBee due to its high-efficient decoding algorithm. Therefore, PatternBee is more applicable for resource-limited IoT devices.

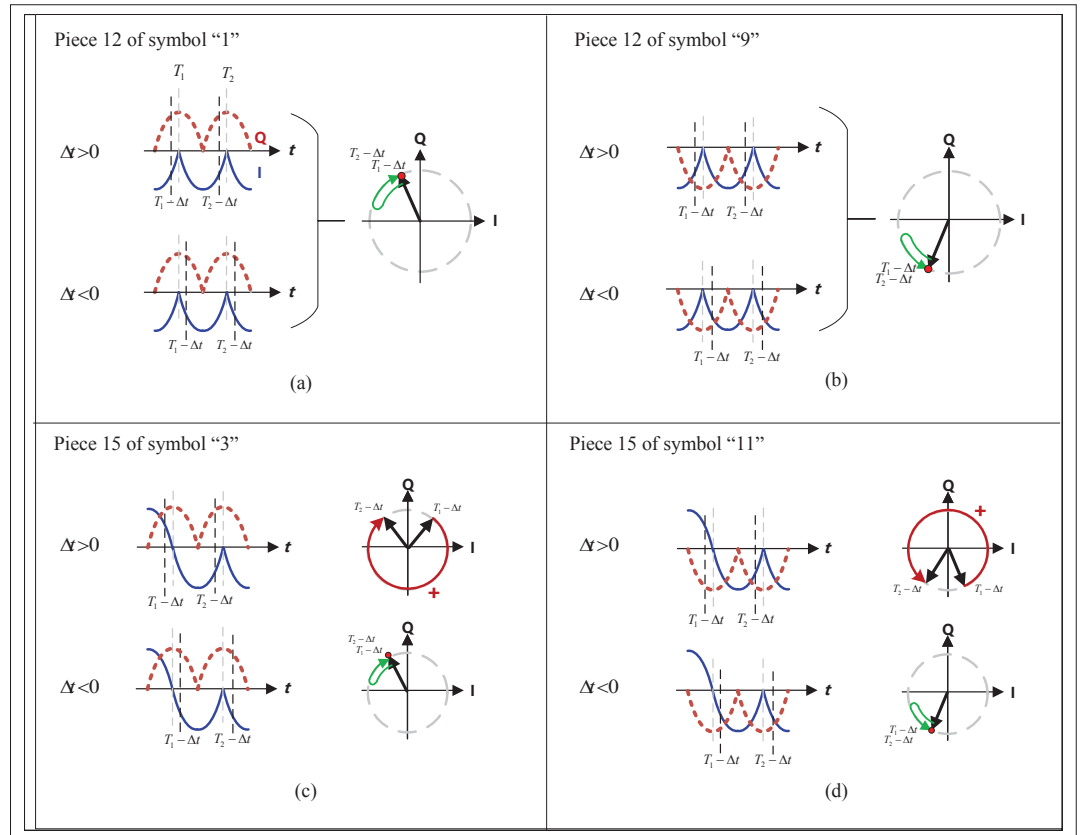


FIGURE 5. a) The piece 12 of ZigBee symbol “1” is a full-ORP and appears in the 2nd quadrant; b) the piece 12 of ZigBee symbol “9” is a full-ORP and appears in the 3rd quadrant; c) the piece 15 of the ZigBee symbol “3” is a semi-ORP because its phase shift is 0 only when $\Delta t < 0$ and appears in the 2nd quadrant; d) the piece 15 of the ZigBee symbol “11” is a semi-ORP because its phase shift is 0 only when $\Delta t < 0$ and appears in the 3rd quadrant.

that in the presence of Δt , some pieces have a phase shift of 0, while others have not. We define a piece with phase shift of 0 to be an Offset Resistant Piece (ORP). There are two types of ORPs:

- **Full-ORP:** A full-ORP is a piece with phase shift of 0, whether $\Delta t < 0$ or $\Delta t > 0$. Figure 4 shows all full-ORPs (marked by red solid boxes) for all ZigBee symbols. Figure 5a shows that the piece 12 of the ZigBee symbol “1” is a full-ORP, because it has a phase shift of 0 when $\Delta t < 0$ and $\Delta t > 0$.
- **Semi-ORP:** A semi-ORP is a piece with phase shift of 0, either when $\Delta t < 0$ or $\Delta t > 0$. Figure 4 shows some semi-ORPs (marked by red solid circles) of ZigBee symbols “3” and “11.” Figure 5c shows that the piece 15 of the ZigBee symbol “3” is a semi-ORP, because it has a phase shift of 0 when $\Delta t < 0$ but has a positive phase shift when $\Delta t > 0$.

PATTERN OF ORPs

By observing all pieces of all ZigBee symbols, we find that full-ORPs and semi-ORPs exhibit some patterns, as summarized in Table 1. Below we explain these patterns.

Patterns of Full-ORPs: Figure 4 shows all full-ORPs (marked by red solid boxes) for all ZigBee symbols. From this figure, we can see that ZigBee symbols “3” and “11” do not have any full-ORP, and except these two symbols, each ZigBee symbol has only one full-ORP. Let $\Omega = \{2, 4, 6, 8, 10, 12, 14\}$. We further reveal that full-ORPs have the following properties.

- Each full-ORP $i \in \Omega$ is associated with two different symbols. For example, full-ORP 2 is associated with symbols “4” and “12.”
- Given a full-ORP $i \in \Omega$, the quadrant q of its sampling point at the time offset Δt differs for its associated two different symbols, but the value of q is either 2 or 3. For example, for the piece 12, $q = 2$ for symbol “1” and 3 for symbol “9,” as shown in Fig. 5a and b.

Patterns of Semi-ORPs: Note that only symbols “3” and “11” have no full-ORPs. We check their all pieces and find that pieces 15 and 16 are their semi-ORPs, which is marked by red circles in Fig. 4. We further reveal that the two semi-ORPs have the following properties:

- Semi-ORP 15 has a phase shift of 0 when $\Delta t < 0$ for symbols “3” and “11,” but the quadrant q of its sampling point at the time offset Δt is 2 for symbols “3” and 3 for symbols “11,” as shown in Figs. 5c and 5d.
- Semi-ORP 16 has a phase shift of 0 when $\Delta t > 0$ for symbols “3” and “11,” but the quadrant q of its sampling point at the time offset Δt is 2 for symbols “3” and 3 for symbols “11.”

PATTERN-BASED DECODING ALGORITHM

Figure 4 and Table 1 above show that each ZigBee symbol consisting of $n = 16$ pieces can be identified by the index and quadrant of its unique ORP. Hence, to decode a symbol, the proposed pattern-based algorithm at most checks each of its n pieces once to find the ORP, and then deter-

Symbol \ ORP Index (i)	2	4	6	8	10	12	14	15 ($\Delta t < 0$)	16 ($\Delta t > 0$)
Quadrant (q)									
2	4	5	6	7	0	1	2	3	3
3	12	13	14	15	8	9	10	11	11

TABLE 1. Pattern of ORPs.

mines the index i and quadrant q of the ORP. Hence, the time complexity of our algorithm is $O(n)$.

Now, we are ready to present our decoding algorithm (Algorithm 1). For each received ZigBee symbol, let $\mathbf{C} = \{(I_i, Q_i)\}_{i=1}^{17}$ denote a sample set of its pieces (from the quadrature demodulator module in Fig. 3) and let $\Delta\Phi = \{\Delta\Phi_i\}_{i=1}^{16}$ denote the phase shifts of these pieces (from the phase shift module in Fig. 3). Assume that we measure the sign of Δt (say, using frame headers [8]) and let $x = 15$ if $\Delta t > 0$ and 16 otherwise. Algorithm 1 uses $\mathbf{C}$, $\Delta\Phi$, and x as inputs (line 2) and returns the identified ZigBee symbol index S as an output. Below, we explain the core part of the algorithm, that is, to determine the index i and quadrant q of the ORP (lines 5 and 6) and decode the ZigBee symbol (lines 7 to 8).

For each $i \in \{2, 4, \dots, 14, x\}$, we process piece i as follows:

- Line 5 to 6: We check if $|\Delta\Phi_i|$ is smaller than the predefined threshold. If yes, we obtain the quadrant q of piece i by constellation point of (I_i, Q_i) .
- Line 7 to 8: If $q = 2$ or 3, we can safely infer that pieces i is an ORP because these ORP only appear in quadrants 2 and 3. Therefore, we can return the ZigBee symbol index S according to Table 1 $\langle i, q \rangle$.

PROCESSING COMPLEXITY

Recall that a ZigBee symbol can be divided into n pieces. In PatternBee, when receiving an n -piece ZigBee symbol, the BLE receiver identifies the symbol by scanning these n pieces and detecting their occurrence pattern. Therefore, the time complexity of PatternBee is $O(n)$.

In contrast, in XBee, an n -piece ZigBee symbol is transformed to an n -bit ideal sequence; in total, there are n corresponding n -bit ideal sequences. When receiving an n -piece ZigBee symbol, the BLE receiver first transforms it into an n -bit sequence and then identifies the symbol by exhaustively correlating the transformed n -bit sequence with each of n n -bit ideal sequences. Therefore, the time complexity of XBee is $O(n^2)$, which is far higher than that of PatternBee.

EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of PatternBee via hardware experiment.

In our experiment, we adopt three universal software radio peripheral B200 (USRP-B200) boards to implement PatternBee and XBee, where XBee is used as a baseline for comparison. As shown in Fig. 6a, we use one board for the ZigBee transmitter, another for the PatternBee receiver (where the decoding threshold in line 5 of Algorithm 1 is set to 0.2), and the other for the XBee receiver. In the experiment, the ZigBee transmitter sends 30,000 ZigBee packets to one receiver

```

1: /*  $x \leftarrow 15$  if  $\Delta t > 0$ ;  $x \leftarrow 16$  if  $\Delta t < 0$  */
2: Input:  $\mathbf{C} = \{(I_i, Q_i)\}_{i=1}^{17}$ ,  $\Delta\Phi = \{\Delta\Phi_i\}_{i=1}^{16}$ ,  $x$ 
3: Output: ZigBee symbol index  $S$ 
4: for  $i$  from  $\{2: 2: 14, x\}$ 
5:   if  $|\Delta\Phi_i| < \text{threshold}$ 
6:      $q \leftarrow \text{quadrant}(I_i, Q_i)$ ;
7:     if  $q = 2$  or  $3$ 
8:       return  $S \leftarrow \text{Table 1 } \langle i, q \rangle$ ;
9:   end
10: end
11: end

```

ALGORITHM 1. Pattern-based decoding.

(i.e., either PatternBee or XBee receiver), where the transmission power is set to 0dBm, the transmission interval is set to 0.01s, and each packet consists of a 32-bit preamble and a default payload of 25 bytes (which is the typical payload length for ZigBee communications [14]).

In our performance evaluation, we compare PatternBee with XBee, in terms of throughput (i.e., the number of successfully received bits per unit time), decoding time complexity (DTC for short, that is, the time of decoding a packet), and energy consumption (measured in μJ , that is, the consumed energy of decoding a packet). Below, we present the evaluation results, as shown in Figs. 6b–6d.

Figure 6b plots the throughputs of PatternBee and XBee, as the transmitter-receiver distance increases from 50cm to 450cm. From this figure, we have the following observations:

- In a short range ($< 150\text{cm}$), PatternBee's throughput is up to 56.3 percent higher than XBee's throughput. Further, the former decreases with the distance, because the signal-to-noise ratio (SNR) decreases with the distance. In contrast, the latter increases with the distance, because XBee decodes packets by counterintuitively leveraging the sampling offset due to signal distortion, which is insignificant within a short range.
- In a middle range (150cm–450cm), PatternBee's throughput is roughly equal to (i.e., only 0.67 percent lower than) XBee's throughput. Further, their throughputs remain almost unchanged, implying that their decoding performance is insensitive in this range.
- In a long range ($> 450\text{cm}$), PatternBee's throughput is 18 percent lower than XBee's throughput. Further, their throughputs decrease with the distance, because the signal quickly attenuates in this range and the SNR becomes low, which is the dominating factor governing the decoding performance.

PatternBee enables the BLE receiver to quickly decode ZigBee symbols only by checking their pieces. Compared to the state-of-the-art RX-side CTC design, PatternBee achieves high decoding accuracy with low processing complexity. In the future, we will extend the communication distance of PatternBee further.

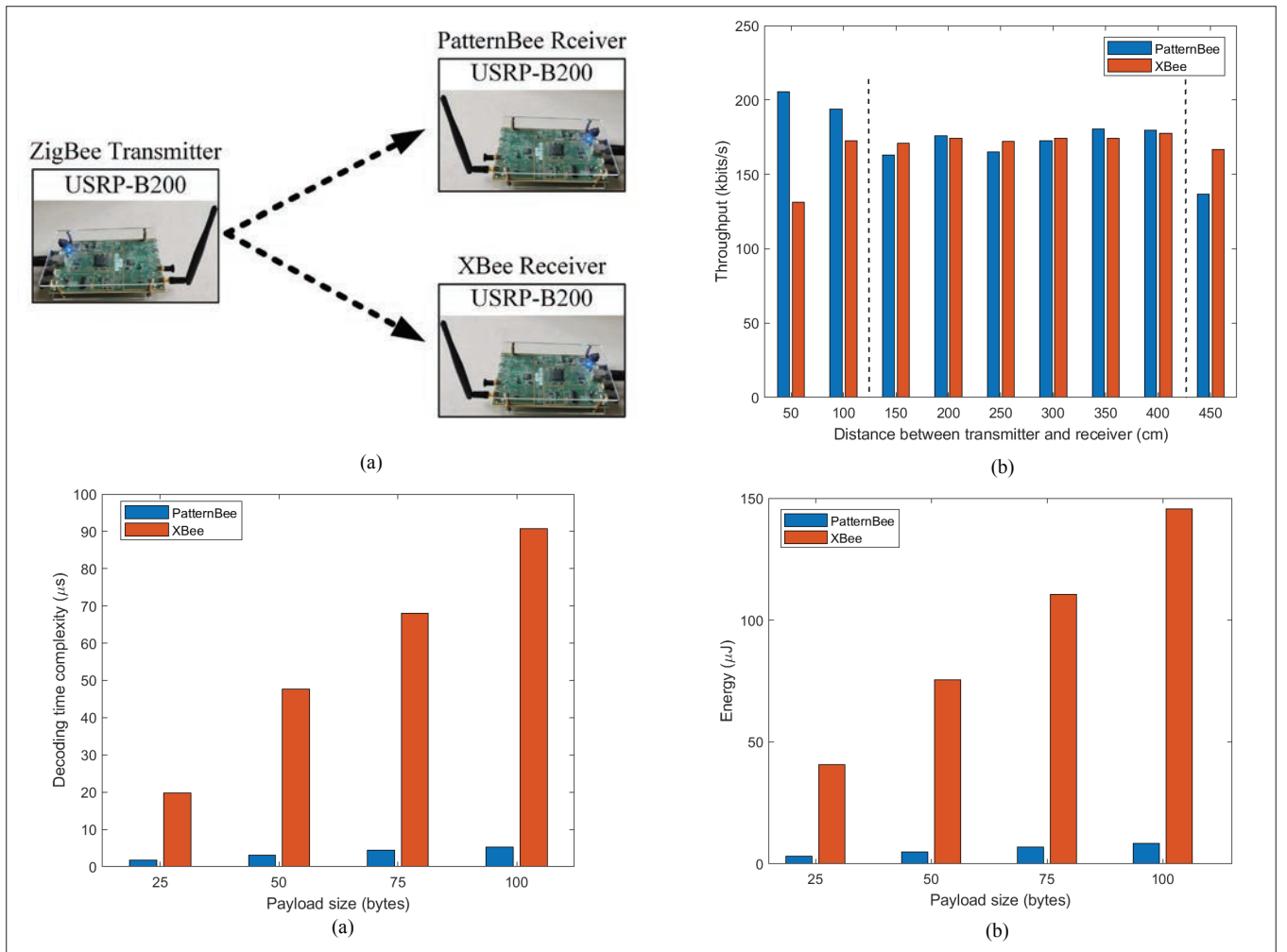


FIGURE 6. Hardware experiments: a) experiment settings; b) throughput vs. distance; c) DTC vs. payload size; d) energy consumption vs. payload size.

Figure 6c plots the DTC of PatternBee and XBee, as the frame payload size increases from 25 bytes to 100 bytes, where the transmitter-receiver distance is set to 150cm. We can see that although both the DTCs of PatternBee and XBee increase with the packet size, the DTC of PatternBee is 93.75 percent lower than that of XBee, because PatternBee does not need to perform exhaustive correlation like XBee.

Figure 6d plots the energy consumption of PatternBee and XBee, under the same experiment setting of Fig. 6(c). It has been pointed out in [8] that when the decoding algorithm is implemented in the CC2650-BLE board, on average decoding one ZigBee symbol will take about 174 μ s and the corresponding energy consumption is about 0.7 μ J. Then utilizing the time-energy relationship and the decoding time in Fig. 6(c), we can calculate the energy consumption. From this figure, we can see that the energy consumption of PatternBee is 93.75 percent lower than that of XBee.

In summary, while PatternBee achieves almost the same throughput as XBee, it has much lower decoding time complexity and energy consumption than XBee due to its high-efficient decoding algorithm. Therefore, PatternBee is more applicable for resource-limited IoT devices.

CONCLUSION

In this article, we present PatternBee, a new RX-side PHY-level CTC for ZigBee-to-BLE direct communications. PatternBee reuses standard BLE hardware modules and meanwhile introduces a software-defined module. This module fully utilizes the occurrence pattern of offset resist pieces to identify ZigBee symbols. As a result, PatternBee enables the BLE receiver to quickly decode ZigBee symbols only by checking their pieces. Compared to the state-of-the-art RX-side CTC design, PatternBee achieves high decoding accuracy with low processing complexity. In the future, we will extend the communication distance of PatternBee further.

ACKNOWLEDGMENT

This work is funded in part by the National Natural Science Foundation of China (File no. 61872451, 61872452, U1911401, and 61727810), and in part by the Science and Technology Development Fund, Macau SAR (File no. 0098/2018/A3, 0037/2020/A1, and 0062/2020/A2).

REFERENCE

- [1] "50B IoT Devices Connected by 2020 — Beyond the Hype and into Reality | RCRWireless | Internet of Things and M2M." <https://iot.williamgraham.ca/iot/50b-iot-devices-connected-by-2020-beyond-the-hype-and-into-reality-rcrwireless/> (accessed July 07, 2020).

- [2] T. Zachariah *et al.*, "The Internet of Things Has a Gateway Problem," *Proc. 16th Int'l. Workshop on Mobile Computing Systems and Applications*, 2015, pp. 27–32.
- [3] X. Zheng, Y. He, and X. Guo, "StripComm: Interference-Resilient Cross-Technology Communication in Coexisting Environments," *Proc. IEEE INFOCOM 2018-IEEE Conf. Computer Commun.*, 2018, pp. 171–79.
- [4] X. Guo, X. Zheng, and H. Yuan, "WiZig: Cross-Technology Energy Communication over a Noisy Channel," 2017.
- [5] K. Chebrolu and A. Dhekne, "Esense: Communication through Energy Sensing," 2009.
- [6] S. M. Kim and T. He, "FreeBee: Cross-Technology Communication via Free Side-channel," 2015, pp. 317–30.
- [7] Z. Li and T. He, "Webee: Physical-Layer Cross-Technology Communication via Emulation," *Proc. 23rd Annual Int'l. Conf. Mobile Computing and Networking*, 2017, pp. 2–14.
- [8] W. Jiang *et al.*, "Achieving Receiver-Side Cross-Technology Communication with Cross-Decoding," *Proc. 24th Annu. Int. Conf. Mob. Comput. Netw.*, Oct. 2018, available: <http://par.nsf.gov/biblio/10076615>.
- [9] H. He *et al.*, "Reliable Cross-Technology Communication with Physical-Layer Acknowledgement," *IEEE Trans. Commun.*, vol. 68, no. 8, 2020, pp. 5175–87.
- [10] S. Wang, S. M. Kim, and T. He, "Symbol-Level Cross-Technology Communication via Payload Encoding," *Proc. 2018 IEEE 38th Int'l. Conf. Distributed Computing Systems (ICDCS)*, 2018, pp. 500–10.
- [11] X. Guo *et al.*, "LEGO-Fi: Transmitter-Transparent CTC with Cross-Demapping," *Proc. IEEE INFOCOM 2019 – IEEE Conf. Computer Commun.*, 2019, pp. 2125–33.
- [12] G. Ballou, "Handbook for Sound Engineers," 2005, <https://books.google.com/books?id=y0d9VA0lkogC>.
- [13] S. Gronemeyer and A. McBride, "MSK and Offset QPSK Modulation," *IEEE Trans. Commun.*, vol. 24, no. 8, 1976, pp. 809–20.
- [14] Z. Li and T. He, "Webee: Physical-Layer Cross-Technology Communication via Emulation," *Proc. 23rd Annual Int'l. Conf. Mobile Computing and Networking*, 2017, pp. 2–14.

BIOGRAPHIES

SHUMIN YAO (roller44@163.com) received the B.S. degree from Beijing Institute of Technology, Zhuhai, China, in 2016, and the M.S. degree from Macau University of Science and Technology (MUST), Macau, China, in 2018. He is currently working toward the Ph.D. degree at MUST. His current research interests include wireless LAN and Internet of things.

LI FENG (lfeng@must.edu.mo) received the M.S. degree from the Department of Mathematics, the University of Hong Kong, Hong Kong in 2007, and the Ph.D. degree from the Faculty of Information Technology (FIT), Macau University of Science and Technology (MUST), Macao in 2013. She is now an associate professor in FIT, MUST. Her research interests include wireless and mobile networks, power saving, SDN, and network performance analysis.

JING ZHAO (1809853jii20008@student.must.edu.mo) received the B.S. degree from Huaiyin Normal University, Huaiyin, China, in 2018, and the M.S. degree from Macau University of Science and Technology (MUST), Macau, China, in 2020. Her current research interests include wireless LAN and Internet of Things.

QINGLIN ZHAO (zqlct@hotmail.com) received the Ph.D. degree from the Chinese Academy of Sciences, Beijing, China, in 2005. From May 2005 to August 2009, he did research with The Chinese University of Hong Kong, Shatin, Hong Kong, and then with The Hong Kong University of Science and Technology, Sai Kung, Hong Kong. Since September 2009, he has been with the Faculty of Information Technology, Macau University of Science and Technology, Macau, China. His research interests include wireless communications and networking, next generation wireless LANs, software defined wireless networking, and Internet of Things.

QIYU YANG (yangqiyu@gdut.edu.cn) received the B.S. and M.S. degrees in automatic engineering from Huazhong University of Science and Technology, Wuhan, China, in 2001 and 2004, respectively, and the Ph.D. degree in control science and engineering from Guangdong University of Technology, in 2015. He has been a lecturer in the School of Automation, Guangdong University of Technology, since 2006. His research interests include modeling of complex systems, signal processing, biomedical devices, and real time data processing.

WENCHAO JIANG (wenchao_jiang@sutd.edu.sg) received his Ph.D. degree from the Department of Computer Science, University of Minnesota, Twin Cities. He is an assistant professor at the Singapore University of Technology and Design (SUTD). His research interests include cross-technology communication, cyber-physical systems, Internet of Things, wireless networks and related areas. His works appear on top conferences and journals such as MobiCom, SenSys, INFOCOM and TON. He has been a topic editor for the *Electronics Journal* since 2020.